

UNIVERSIDAD DEL VALLE DE GUATEMALA

Inga. Kimberly Barrera



Parcial 1 - Consultoría HPC Usando OpenMP

Bryan Alberto Martínez Orellana 23542
Adriana Sophia Palacios Contreras 23044

GUATEMALA
11 de septiembre de 2026

Repositorio

https://github.com/Bamo0507/Parcial_1_Paralela_Rutas

Contexto y Datos

Descripción del problema

Para este proyecto se estuvo trabajando desde la consultoría de Paralelink, empresa que se dedica principalmente al análisis de algoritmos aplicados sobre datos que se representan por medio de grafos, es decir, estructuras en donde se tienen nodos que representan elementos dentro de un esquema y aristas que representan las relaciones entre estos elementos. Bajo este escenario, se nos presentó el caso de una empresa que desea hacer un análisis sobre su red social, en donde cada usuario es un nodo y cada amistad es una arista, y lo que se busca es encontrar de forma óptima el camino más corto de conexiones entre dos individuos cualesquiera.

El inconveniente principal, como sucede en toda red social, es que la distribución de amistades es fuertemente desbalanceada: la gran mayoría de las personas tiene una cantidad de amigos que se mueve en las unidades, decenas o a lo sumo centenas, mientras que existe un grupo muy reducido de usuarios que llega a tener decenas de miles de amistades. Esto representa un problema a la hora de repartir el trabajo, ya que si la exploración se divide de forma ingenua, puede darse el caso en el que un trabajador termine en un instante porque le tocó una persona con dos amigos, mientras que otro se queda estancado recorriendo decenas de miles de vecinos porque le tocó uno de estos usuarios altamente conectados. Es por esto que el problema no es únicamente encontrar el camino más corto, sino hacerlo de tal forma que el trabajo quede balanceado entre los trabajadores sin importar con qué tipo de usuario les toque lidiar.

Propuesta secuencial

A la hora de discutir el problema y definir la propuesta para el algoritmo secuencial, lo primero que se nos vino a la mente fue una solución de fuerza bruta, en donde se plasmara el grafo en una especie de matriz de adyacencia y se recorriera llevando el conteo de los saltos, de tal forma que cada celda con una conexión representara la relación de amistad entre dos personas. No obstante, esta idea quedó descartada rápidamente, ya que al estar trabajando con millones de usuarios una matriz de ese tamaño no solo sería inviable en memoria, sino que

además implicaría recorrer una cantidad enorme de celdas vacías, dado que cada persona se relaciona únicamente con una fracción mínima del total. En otras palabras, no era una solución eficiente para el tamaño del problema.

Dicho esto, se comenzaron a evaluar los distintos algoritmos conocidos para recorrer grafos y encontrar rutas mínimas. Los primeros en quedar descartados fueron todos aquellos que dependen de pesos en las aristas, entre los cuales están Dijkstra y A*, que justamente sirven para encontrar rutas más cortas pero necesitan una medida del costo de cada conexión. En este caso ese peso no existe, porque no hay forma de decir que una amistad vale más que otra, una amistad es una amistad y la relación es directa, es decir, el grafo no tiene pesos y aplicar estos algoritmos únicamente agregaría el costo de mantener una cola de prioridad sin aportar nada a la respuesta.

Una vez descartados, nos quedamos con el BFS, que al explorar el grafo por niveles garantiza que la primera vez que se alcanza al usuario destino se hace con la cantidad mínima de saltos. En lugar de una cola tradicional, la frontera de cada nivel se mantiene como un arreglo, se recorren todos sus nodos marcando la distancia y el padre de cada vecino no visitado, y con esos vecinos se construye la frontera del siguiente nivel. Al encontrar al destino se termina el nivel en curso y se detiene la búsqueda, y con el arreglo de padres se reconstruye el camino. Se eligió esta estructura por niveles desde el secuencial, y no la cola clásica, para que la versión paralela fuera exactamente el mismo algoritmo con el mismo criterio de terminación, de tal forma que la comparación de tiempos entre ambas sea justa.

Cabe mencionar que esta estructura sigue siendo la cola de tareas pendientes que plantea el problema, solo que implementada de otra forma. En un BFS con una cola tradicional, en cualquier momento la cola tiene dos grupos de nodos: los del nivel actual que todavía faltan por procesar, y los del siguiente nivel que ya se descubrieron. Lo que hicimos fue guardar esos dos grupos en dos arreglos separados en lugar de una sola cola. El orden en que se exploran los nodos y las distancias que se obtienen son exactamente los mismos, y lo comprobamos comparando la cantidad de nodos visitados entre ambas versiones para cada consulta. Se tomó esta decisión desde el secuencial porque una cola compartida entre threads obliga a poner un candado en cada inserción y extracción, y con decenas de millones de aristas los threads pasarían más tiempo esperando el candado que explorando, de tal forma que la versión paralela terminaría siendo

más lenta que la secuencial. Separar la cola por niveles es lo que permite repartir cada nivel entre los threads sin ese bloqueo.

Justificación de los datos de prueba

En cuanto al origen de los datos, se decidió construir un generador propio en Python en lugar de utilizar un dataset público, esto garantizó que ambos integrantes utilizáramos el mismo grafo a partir de una semilla fija (123), sin necesidad de versionar un archivo de más de 500 MB que GitHub no permite subir. Para que los datos se sintieran reales, cada usuario recibe un nombre único formado por un nombre y dos apellidos tomados de listas de 200 opciones cada una, lo que da 8,000,000 de combinaciones posibles. Estos se pueden encontrar en `/generador/nombres.txt` y `/generador/apellidos.txt`.

Respecto al tamaño de la muestra, el grafo cuenta con 2,000,000 de usuarios y 37,276,447 amistades, con una cantidad de relaciones promedio de 37 y un máximo de 33,532 en un único nodo. La distribución de amistades se asignó por buckets con pesos, buscando imitar la forma de una red social real: el 80 % de los usuarios tiene entre 1 y 9 amigos, el 17.5 % entre 10 y 99, el 2.3 % entre 100 y 999, el 0.19 % entre 1,000 y 9,999, y únicamente el 0.01 %, es decir unos 200 usuarios, tiene entre 10,000 y 50,000 amistades. Con esto se obtiene una razón de aproximadamente 5,000 veces entre el usuario menos y más conectado, que es exactamente el desbalance que el problema plantea.

Por último, en cuanto a las estructuras de datos en memoria, el grafo se almacena en formato CSR (Compressed Sparse Row), que consiste en dos arreglos planos: uno de vecinos, en donde las listas de amigos de todos los usuarios están pegadas una tras otra sin huecos, y uno de índices de tamaño $N+1$ que indica en qué posición empieza la lista de cada usuario, de tal forma que los amigos del usuario "u" son `"vecinos[indices[u] .. indices[u+1])"` y su grado se obtiene en complejidad constante como la diferencia entre ambos índices. Además, el CSR es de solo lectura durante toda la búsqueda, lo que en la versión paralela permite que todos los threads lo consulten sin ningún tipo de bloqueo. En total, el grafo ocupa aproximadamente 314 MB en RAM.

Estrategia de Paralelización

Directivas de OpenMP utilizadas y por qué

La versión paralela conserva la misma estructura por niveles del secuencial, y lo que se paraleliza es el recorrido de la frontera de cada nivel, para esto se utilizaron cuatro directivas.

Primero, `#pragma omp parallel` abre una región paralela por cada nivel del BFS. Dentro de esta región, cada thread obtiene su identificador con `omp_get_thread_num()` y toma su propio buffer, en donde va guardando los nodos que descubre sin tocar los buffers de los demás. Se abre una región por nivel y no una sola para todo el BFS porque entre un nivel y el siguiente hay que saber cuántos nodos se descubrieron en total, y esa información solo existe cuando todos los threads terminaron.

Luego, `#pragma omp for schedule(runtime) nowait` es la directiva que reparte el trabajo: divide las posiciones de la frontera entre los threads, y cada thread recorre los vecinos de los nodos que le tocaron. Se usa `schedule(runtime)` para poder cambiar la estrategia de reparto con la variable de entorno `OMP_SCHEDULE` sin recompilar, lo cual nos permitió comparar `static`, `guided` y `dynamic` con distintos tamaños de bloque usando el mismo binario. El `nowait` quita la barrera implícita del `for`, ya que justo después ponemos una barrera nuestra en el punto donde realmente la necesitamos, y de esta forma no se paga dos veces.

Después, `#pragma omp barrier` hace que todos los threads esperen a que el último termine de recorrer su parte de la frontera. Es necesaria porque el siguiente paso, armar la frontera del nuevo nivel, depende de que ya nadie esté escribiendo en su buffer.

Finalmente, `#pragma omp single` hace que un solo thread calcule el prefix sum sobre los contadores de cada buffer, es decir, en qué posición de la nueva frontera va a copiar cada thread sus nodos descubiertos. Es un cálculo pequeño (un ciclo de tantas iteraciones como threads hay) y no vale la pena repartirlo. El `single` trae su propia barrera al final, de tal forma que cuando los threads salen de ahí ya todos conocen su posición y copian su buffer a la frontera en paralelo, cada uno en su rango, sin que se estorben.

Cabe mencionar que decidimos no usar `#pragma omp critical` ni `#pragma omp task`. El `critical` habría sido la forma más directa de proteger la marca de visitado, pero serializa a todos los threads en cada descubrimiento, y en un grafo con 74 millones de arcos eso destruye cualquier ganancia. Los task se acercan más a la idea de "cola de tareas pendientes", pero crear una tarea por nodo tiene un costo fijo que, con millones de nodos se consideró era demasiado como para valer la pena.

Cómo se evitaron las condiciones de carrera

En este algoritmo hay tres puntos donde varios threads podrían escribir sobre lo mismo, y cada uno se resolvió de forma distinta.

El primero y más importante es la marca de visitado. Dos threads pueden descubrir al mismo nodo al mismo tiempo desde dos padres distintos, y si ambos lo meten a la siguiente frontera, ese nodo se procesa dos veces y el trabajo duplicado se propaga a los niveles siguientes. Para evitarlo se usa una operación atómica de `compare-and-swap` sobre el arreglo de distancias: cada thread intenta cambiar la distancia del vecino de -1 al nivel actual, y solo el thread que logra hacer el cambio es el que guarda el padre y agrega el nodo a su buffer. El otro thread ve que el valor ya no es -1 y simplemente sigue. De esta forma no hay candados, y cada nodo entra a la frontera exactamente una vez. Adicional, antes de intentar el `compare-and-swap` se hace una lectura simple de la distancia, porque en un grafo social la gran mayoría de las aristas apuntan a nodos que ya fueron visitados, y con esa lectura se descarta el caso común sin pagar el costo de la operación atómica.

El segundo punto es la construcción de la siguiente frontera. Si todos los threads escribieran en un mismo arreglo compartido, habría que proteger cada escritura. En lugar de eso, cada thread tiene su propio buffer, y solo al cerrar el nivel se calcula con el prefix sum en qué parte de la frontera final va cada buffer. Así, durante el recorrido nadie comparte nada, y en la copia final cada thread escribe en un rango que es solo suyo.

El tercer punto es el contador de nodos descubiertos por thread. Aunque cada thread tenga el suyo, si los contadores están uno al lado del otro en memoria caen en la misma línea de caché, y cada vez que un thread escribe el suyo invalida la línea para los demás. Para evitarlo, cada thread acumula su conteo en una variable local durante todo el nivel y lo escribe una sola vez al final.

Cómo se atacó el desbalance de carga

El desbalance en este problema viene de que un nodo de la frontera puede tener 2 vecinos y otro puede tener 33,000. Si el reparto de la frontera es static, cada thread recibe un bloque fijo de posiciones desde el inicio, y al que le tocan los nodos muy conectados termina trabajando solo mientras los demás esperan en la barrera. Es por esto que la estrategia principal fue `schedule(dynamic)`, en donde los threads van pidiendo bloques de la frontera conforme se desocupan, de tal forma que si uno se atrasa con un nodo pesado, los demás siguen tomando el trabajo que queda.

Dicho esto, `dynamic` por sí solo no basta, el tamaño del bloque importa igual. Para que `dynamic` pueda rebalancear tiene que haber muchos más bloques que threads, porque si la frontera se divide en 10 bloques para 10 threads, cada uno toma el suyo y no queda nada por redistribuir, que es básicamente lo mismo que `static`. Por otro lado, con bloques de tamaño 1 cada nodo de la frontera implica un incremento atómico sobre el contador compartido del ciclo, y en niveles con cientos de miles de nodos ese costo se vuelve mayor que lo que se gana en balance. En las mediciones esto se vio claramente: `dynamic,1` fue la peor configuración de todas, incluso peor que `static`, mientras que bloques entre 16 y 1024 quedaron todos dentro de un rango cercano, con el mejor resultado en `dynamic,64` en una máquina y `dynamic,256` en la otra. No fijamos el tamaño de bloque por intuición, se midió con `OMP_SCHEDULE` sobre el mismo binario y se eligió el que dio la menor mediana.

Por último, hay que reconocer un límite de este enfoque: el reparto del for divide la frontera por nodos, y ningún scheduling puede partir el trabajo de un solo nodo. Si un usuario con 33,000 amigos cae en un bloque, el thread que lo tomó tiene que recorrer esos 33,000 vecinos solo. `dynamic` reduce el problema porque los demás threads siguen trabajando mientras tanto, pero ese nodo igual marca el final del nivel. Resolverlo del todo implicaría repartir el trabajo por aristas en lugar de por nodos, lo cual podría ser una opción a probar en otra oportunidad.

Resultados y Métricas

Para medir, no se cronometró una sola búsqueda, sino un lote de 100 búsquedas. Es decir, se generaron 100 pares de usuarios (un origen y un destino cada uno) de forma aleatoria con una semilla fija, se guardaron en un archivo dentro del repositorio, y lo que se mide es el tiempo que tarda el programa en encontrar el camino más corto para los 100 pares, uno tras otro. Se hizo así por dos razones: primero, porque una sola búsqueda toma entre 0.06 y 0.27 segundos según qué tan lejos estén los dos usuarios, y con tiempos tan cortos el ruido pesa demasiado; y segundo, porque al ser los mismos 100 pares para el secuencial, para el paralelo y para ambos integrantes, todas las versiones hacen exactamente el mismo trabajo y las comparaciones son justas. El tiempo reportado no incluye la carga del grafo a memoria, que se hace una sola vez antes de empezar a medir.

Cada configuración se corrió 12 veces seguidas, se descartaron las 2 primeras como calentamiento (en el secuencial de Adriana las primeras cuatro repeticiones tardaron entre 15.5 y 17.5 s contra 7.4 s las demás, porque el grafo todavía no estaba completamente en memoria; dos de esas quedaron dentro de las 10 medidas, pero al usar mediana no afectaron el resultado, que es precisamente la razón de no usar promedio) y se tomó la mediana de las 10 restantes. Se usó mediana en lugar de promedio porque macOS no permite fijar un thread a un núcleo específico, y puede mover uno a un núcleo de eficiencia en cualquier momento, lo cual genera corridas sueltas más lentas que el promedio arrastraría y la mediana no.

El benchmark tiene dos fases. En la primera se corren siete estrategias de scheduling con los 10 threads de la máquina, para elegir la que da la menor mediana: static, que reparte la frontera en bloques fijos desde el inicio; guided, que entrega bloques grandes al principio y cada vez más pequeños conforme se acaba el trabajo; y dynamic con cinco tamaños de bloque distintos (1, 16, 64, 256 y 1024), en donde los threads van pidiendo bloques de ese tamaño conforme se desocupan. Se incluyeron varios tamaños de dynamic porque, como se explicó en la estrategia, el balance y el costo de pedir trabajo dependen directamente de ese número. En la segunda fase se mide con la estrategia ganadora usando 1, 2, 4, 6, 8 y 10 threads, para ver cómo escala. El speedup se calcula como el tiempo del secuencial dividido entre el tiempo del paralelo, y la eficiencia como el speedup dividido entre la cantidad de threads. Adicional, en cada corrida se verificó que la suma de las distancias encontradas para los 100 pares fuera idéntica a la del secuencial, como comprobación de que el paralelo da los mismos resultados.

Ambos integrantes corrieron en una Apple M4 de 10 núcleos, 4 de rendimiento y 6 de eficiencia, lo cual es importante para interpretar los resultados porque los núcleos de eficiencia son considerablemente más lentos.

Ejecución Bryan

```
@ * Documents/Compu_Paralela/Parcial_1_Paralela_Rutas
cat bench/resultados/Bryan maquina.txt bench/resultados/Bryan salida.log
quien: Bryan
fecha: 2026-09-10 23:46:20
sistema: Darwin 24.1.0 arm64
cpu: Apple M4
threads maximos: 10
compilador: Apple clang version 16.0.0 (clang-1600.0.26.6)
grafo: datos/grafos.txt
pares: bench/pares.txt (100 consultas)
repeticiones: 12 (se descartan las primeras 2)
schedule ganador: dynamic,64
### secuencial threads=1 schedule=static
carga: 1.12 s (2000000 nodos, 37276447 amistades)
pares: 100
repeticion 1: 7.8316 s
repeticion 2: 7.3652 s
repeticion 3: 7.5458 s
repeticion 4: 7.1641 s
repeticion 5: 9.1330 s
repeticion 6: 8.2629 s
repeticion 7: 10.8918 s
repeticion 8: 8.2265 s
repeticion 9: 8.7244 s
repeticion 10: 9.0618 s
repeticion 11: 7.9241 s
repeticion 12: 7.7767 s
suma_distancias: 317

### paralelo threads=10 schedule=static
carga: 1.11 s (2000000 nodos, 37276447 amistades)
threads: 10 schedule: static,0
pares: 100
repeticion 1: 3.3197 s
repeticion 2: 3.2196 s
repeticion 3: 3.1734 s
repeticion 4: 3.2982 s
repeticion 5: 3.2048 s
repeticion 6: 3.1795 s
repeticion 7: 3.1712 s
repeticion 8: 3.3021 s
repeticion 9: 3.1861 s
repeticion 10: 3.1828 s
repeticion 11: 3.4318 s
repeticion 12: 3.3641 s
suma_distancias: 317

### paralelo threads=10 schedule=guided
carga: 0.99 s (2000000 nodos, 37276447 amistades)
threads: 10 schedule: guided,1
pares: 100
repeticion 1: 3.0433 s
repeticion 2: 3.0639 s
repeticion 3: 2.9096 s
repeticion 4: 2.9857 s
repeticion 5: 3.0240 s
repeticion 6: 3.2109 s
repeticion 7: 3.1028 s
repeticion 8: 3.0318 s
repeticion 9: 3.1082 s
repeticion 10: 3.0386 s
repeticion 11: 3.1173 s
repeticion 12: 2.9727 s
suma_distancias: 317
```

El secuencial demoró 8.245 s, tomando eso de base y las estrategias de scheduling utilizadas, estos fueron los resultados:

Métrica	static	guided	dynamic, 1	dynamic, 16	dynamic, 64	dynamic, 256	dynamic, 1024
mediana	3.195	3.035	3.857	3.044	2.910	3.024	3.065
speedup	2.58	2.72	2.14	2.71	2.83	2.73	2.69

Escalamiento de threads con dynamic,64:

Métrica	1	2	4	6	8	10
mediana	8.660	4.557	2.968	2.986	3.014	2.910
speedup	0.95	1.81	2.78	2.76	2.74	2.83
eficiencia	0.95	0.90	0.69	0.46	0.34	0.28

Ejecución Adriana

```
cat bench/resultados/Adriana_maquina.txt bench/resultados/Adriana_salida.log
quien: Adriana
fecha: 2026-09-10 23:55:55
sistema: Darwin 25.6.0 arm64
cpu: Apple M4
threads maximos: 10
compilador: Apple clang version 21.0.0 (clang-2100.1.1.101)
grafo: datos/grafos.txt
pares: bench/pares.txt (100 consultas)
repeticiones: 12 (se descartan las primeras 2)
schedule ganador: dynamic,256
### secuencial threads=1 schedule=static
carga: 1.42 s (2000000 nodos, 37276447 amistades)
pares: 100
repeticion 1: 17.4998 s
repeticion 2: 15.4737 s
repeticion 3: 16.5304 s
repeticion 4: 16.0768 s
repeticion 5: 7.9614 s
repeticion 6: 7.4201 s
repeticion 7: 7.3588 s
repeticion 8: 7.3209 s
repeticion 9: 7.4904 s
repeticion 10: 7.2305 s
repeticion 11: 7.2107 s
repeticion 12: 7.3301 s
suma_distancias: 317
```

El secuencial demoró 7.389 s, tomando eso de base y las estrategias de scheduling utilizadas, estos fueron los resultados:

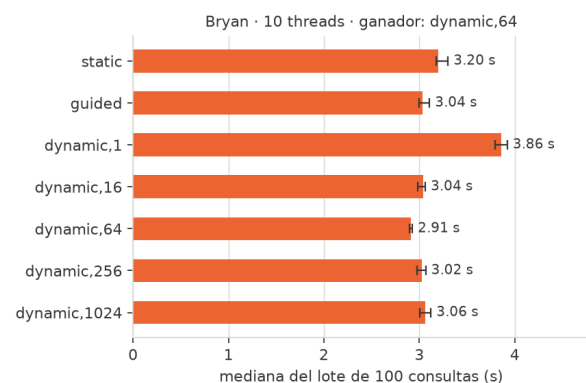
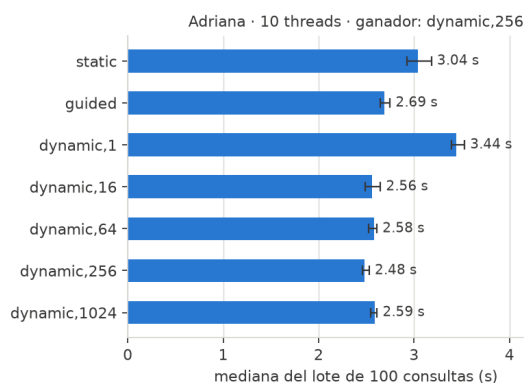
Métrica	static	guided	dynamic, 1	dynamic, 16	dynamic, 64	dynamic, 256	dynamic, 1024
mediana	3.038	2.690	3.441	2.558	2.580	2.484	2.591
speedup	2.43	2.75	2.15	2.89	2.86	2.97	2.85

Escalamiento de threads con dynamic,256:

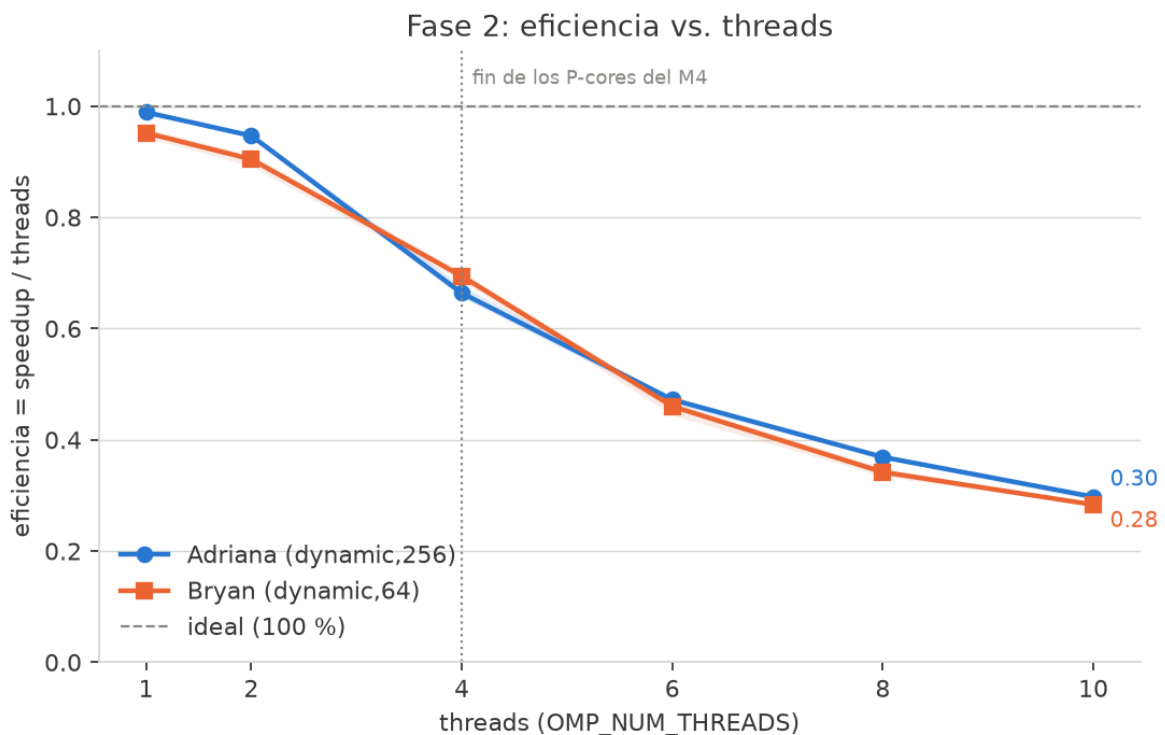
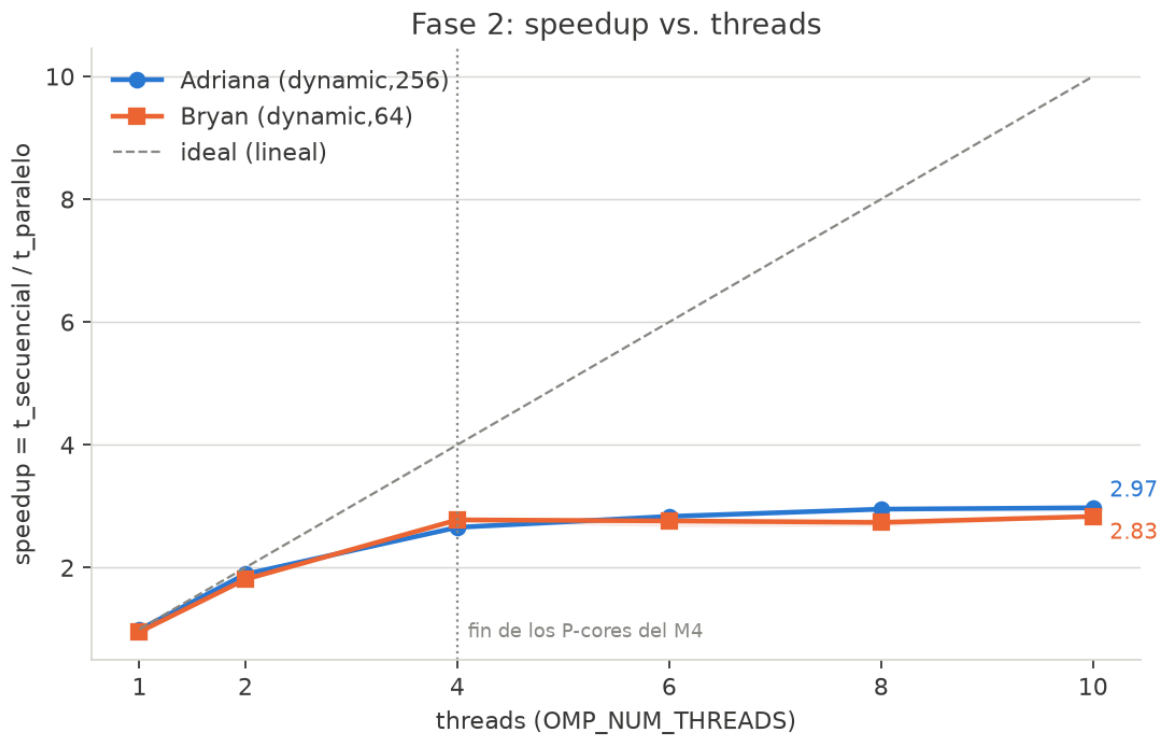
Métrica	1	2	4	6	8	10
mediana	7.471	3.902	2.783	2.608	2.503	2.484
speedup	0.99	1.89	2.66	2.83	2.95	2.97
eficiencia	0.99	0.95	0.66	0.47	0.37	0.30

Análisis

Fase 1: estrategia de scheduling (OMP_SCHEDULE)



Las dos máquinas coinciden en el scheduling. dynamic,1 fue la peor opción en ambas, incluso por debajo de static, porque con bloques de un nodo el costo de pedir trabajo al contador compartido supera lo que se gana en balance. static quedó entre 10 % y 20 % por detrás del mejor dynamic, que es el efecto del desbalance de grados. Entre dynamic,16 y dynamic,1024 las diferencias fueron de alrededor de 3 %, tan pequeñas que el ganador cambió de una máquina a otra.



En el escalamiento, el resultado central es que la eficiencia se mantiene arriba de 0.90 con 2 threads, cae a cerca de 0.68 con 4, y de ahí en adelante el speedup se estanca entre 2.8 y 3.0 sin importar cuántos threads se agreguen. Ese quiebre en 4 coincide con la cantidad de núcleos de rendimiento del M4: hasta 4 threads cada uno corre en un núcleo rápido, y del quinto en adelante caen en núcleos de

eficiencia mucho más lentos. Como cada nivel del BFS termina cuando termina el último thread, un thread lento marca el ritmo de todos, esto es característica del hardware con el que se trabajó.

El paralelo con un solo thread tardó 0.95 y 0.99 del secuencial, es decir, todo el mecanismo adicional (atómicas, buffers por thread, prefix sum, barreras) cuesta entre 1 % y 5 %. La variación entre repeticiones fue menor al 2 % en casi todas las configuraciones, por lo que las bandas casi no se distinguen en las gráficas.

Dicho esto, nuestras decisiones sí mejoraron el secuencial, en un factor cercano a 3, y ese factor está limitado por el hardware y no por el algoritmo. Las tres decisiones que más pesaron fueron separar la cola en frentes por nivel, que permitió paralelizar sin candados; el compare-and-swap con lectura previa, que mantuvo el overhead debajo del 5 %; y el schedule(dynamic) con bloque intermedio, que superó a static entre 10 % y 20 % en ambas máquinas.